

LAB 12

TASK 1:

Code

```
[org 0x0100]
jmp start

name db 'Musawar',0
len dw 7

printstr:
    push bp
    mov bp, sp

    push es
    push si
    push di
    push cx

    mov ax, 0B800h
    mov es, ax

    mov si, [bp+6] ; string address
    mov cx, [bp+4] ; length
    mov ah, 07h   ; normal gray color

next:
    lodsb
    mov [es:di], ax
    add di, 2
    loop next

    pop cx
    pop di
    pop si
    pop es
    pop bp
    ret 4

start:
    ; compute DI = (row*80 + col)*2
    mov al, 80
    mov bl, 10
    mul bl      ; AX = 80*10
    add ax, 20
    shl ax, 1
    mov di, ax

    mov ax, name
    push ax
    push word [len]
```

Explanation

- [org 0x0100] sets the program load address for a COM file.
- jmp start skips the data section and jumps to the code.
- name db 'Musawar',0 stores the string in memory.
- len dw 7 stores the length of the string.
- printstr: defines the string-printing subroutine.
- push bp / mov bp, sp create a stack frame for accessing parameters.
- The code pushes ES, SI, DI, CX to preserve registers.
- mov ax, 0B800h loads video memory segment.
- mov es, ax points ES to video memory.
- mov si, [bp+6] loads the string address.
- mov cx, [bp+4] loads the string length.
- mov ah, 07h sets the character attribute (gray on black).
- lodsb loads one character into AL.
- mov [es:di], ax writes the character and attribute to video memory.
- add di, 2 moves to the next screen cell.
- loop next repeats for all characters.
- Registers are restored and the routine ends with ret 4.
- In start:, the screen offset is calculated using $(\text{row} * 80 + \text{col}) * 2$.
- mov al, 80 loads characters per row.
- mul bl calculates $80 * \text{row}$.
- add ax, 20 adds the column.
- shl ax, 1 converts character index to byte offset.
- mov di, ax sets DI to the target screen position.
- The program pushes string address and length before calling printstr.
- The program exits with INT 21h function 4C00h.

```
call printstr
```

```
mov ax, 4C00h  
int 21h
```

OUTPUT

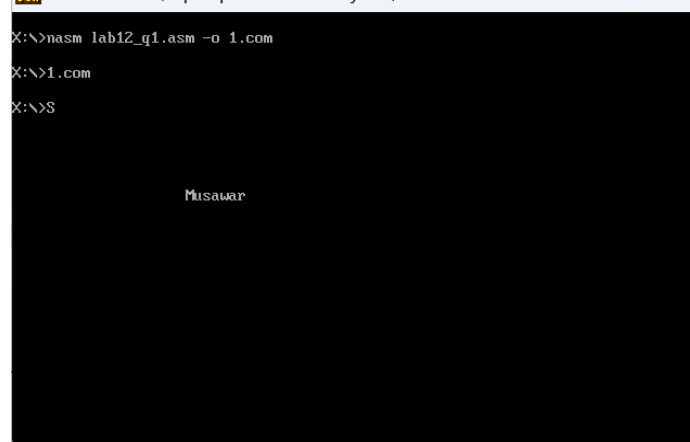


Figure 1

TASK 2:

Code

```
[org 0x0100]  
jmp start  
  
start:  
    mov ax, 0B800h  
    mov es, ax  
  
    ; offset = 5 * 2  
    mov di, 10    ; print at column 5  
  
    mov ax, 43981 ; any number you want  
    mov bx, 16    ; hex base  
    xor cx, cx    ; count digits = 4  
  
repeat:  
    xor dx, dx  
    div bx        ; AX/BX → DX=remainder  
  
    cmp dl, 9  
    jle digit_num  
    add dl, 55    ; A-F  
    jmp push_now  
  
digit_num:  
    add dl, 48    ; 0-9  
  
push_now:  
    push dx
```

Explanation

- [org 0x0100] sets the COM program origin.
- jmp start jumps to the executable code.
- mov ax, 0B800h loads video memory segment.
- mov es, ax enables direct screen writing.
- mov di, 10 sets starting column (5 × 2 bytes).
- mov ax, 43981 loads the number to convert into AX.
- mov bx, 16 sets the base for hexadecimal conversion.
- xor cx, cx clears digit counter for 4 hex digits.
- In each loop:
 1. xor dx, dx clears DX for division.
 2. div bx divides AX by 16 and remainder goes to DL.
- cmp dl, 9 checks whether the digit is 0–9.
- jle digit_num selects the numeric conversion path.
- add dl, 55 converts 10–15 into ASCII A–F.
- digit_num: add dl, 48 converts 0–9 into ASCII.
- push dx stores each digit on the stack (right-to-left order).

```

inc cx
cmp cx, 4
jl repeat

print:
pop dx
mov dh, 07h
mov [es:di], dx
add di, 2
loop print

mov ax, 4C00h
int 21h

```

- inc cx counts digits.
- cmp cx, 4 / jl repeat continues until 4 digits are collected.
- The printing loop pops digits in correct order.
- pop dx retrieves a character + attribute space.
- mov dh, 07h sets attribute.
- mov [es:di], dx writes the digit on screen.
- add di, 2 moves to next column.
- loop print prints all 4 digits.
- The program ends using INT 21h, function 4Ch.

OUTPUT

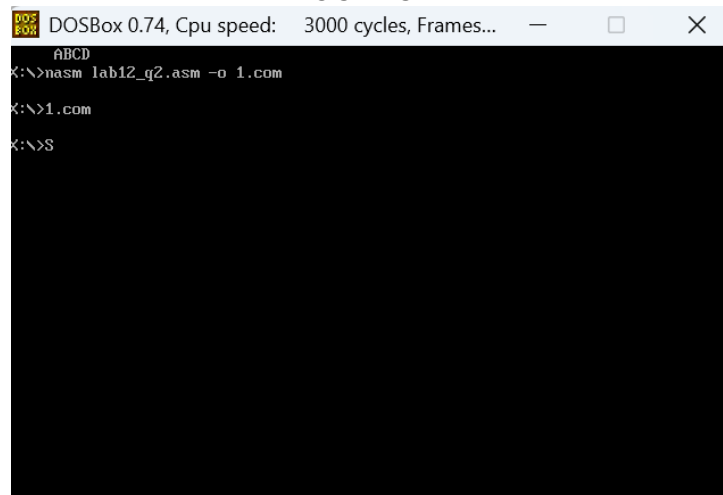


Figure 2

TASK 3:

CODE

```

[org 0x0100]

jmp start

start:
mov ax, 0B800h
mov es, ax

mov al, 80
mov bl, 5
mul bl      ; AX = 5*80
add ax, 10  ; + column
shl ax, 1   ; *2
mov di, ax

mov ax, 0723h ; '#'
mov cx, 30
rep stows

```

EXPLANATION

- [org 0x0100] sets program load origin.
- jmp start jumps over unused bytes.
- mov ax, 0B800h loads video memory segment.
- mov es, ax enables writing directly to video memory.
- **Top Border**
- mov al, 80 loads characters per screen row.
- mov bl, 5 loads starting row.
- mul bl computes 5 * 80.
- add ax, 10 adds the starting column.
- shl ax, 1 converts cell index to byte offset.
- mov di, ax sets DI to top-left position.
- mov ax, 0723h loads character '#' with attribute 07h.
- mov cx, 30 sets width of the top border.
- rep stows writes 30 '#' characters.

```

    mov si, 6      ; start row
middle_rows:
    ; compute offset for left border
    mov ax, 80     ; bytes per row
    mul si         ; AX = row * 80 (si contains row
number)
    add ax, 10     ; column
    shl ax, 1      ; *2 for character+attribute
    mov di, ax

    mov ax, 0723h
    stosw          ; left '#'

    ; move DI to right border
    mov bx, 30
    dec bx         ; 29 positions to move
    shl bx, 1      ; *2
    add di, bx
    mov [es:di], ax ; right '#'

    inc si
    cmp si, 11     ; until row 11
    jle middle_rows

    mov ax, 80
    mov bl, 12
    mul bl         ; AX = 12*80
    add ax, 10     ; + column
    shl ax, 1      ; *2
    mov di, ax

    mov ax, 0723h
    mov cx, 30
    rep stosw

; exit
    mov ax, 4C00h
    int 21h

```

➤ Middle Rows

- mov si, 6 sets starting row for vertical borders.
- In each middle row iteration:
 1. mov ax, 80 loads characters per row.
 2. mul si computes row offset.
 3. add ax, 10 adds column offset.
 4. shl ax, 1 converts to byte address.
 5. mov di, ax sets DI to left border location.
- mov ax, 0723h prepares border character.
- stosw writes left border #.
- For right border:
 1. mov bx, 30 loads width.
 2. dec bx adjusts for rightmost column.
 3. shl bx, 1 converts to bytes.
 4. add di, bx moves DI to right border position.
- mov [es:di], ax writes right border #.
- inc si moves to next row.
- cmp si, 11 / jle repeats until last middle row.
- **Bottom Border**
- mov ax, 80 loads characters per row.
- mov bl, 12 loads bottom row.
- mul bl computes bottom row offset.
- add ax, 10 adds starting column.
- shl ax, 1 converts to byte offset.
- mov di, ax sets DI to start of bottom line.
- mov ax, 0723h loads character and attribute.
- mov cx, 30 loads width.
- rep stosw draws bottom border.
- **Exit**
- mov ax, 4C00h loads DOS terminate function.
- int 21h exits the program.

OUTPUT

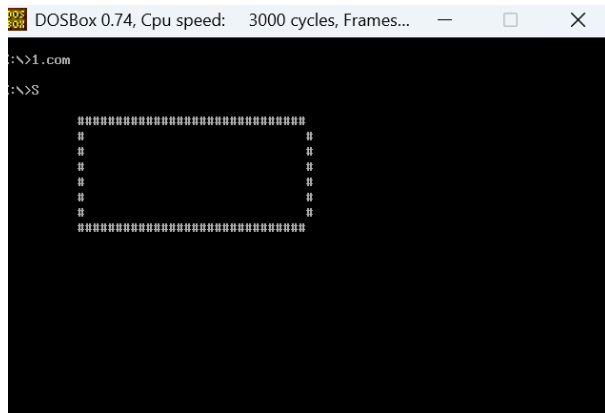


Figure 3